



Jogan · যোগান: an agent liquidity copilot for mobile financial services

Project report · AI Dev Fest 2026 (DIU CPC × upay) · Track 05: Merchant & Agent Intelligence · 4 October 2026

Team Adrenaline, Department of Computer Science and Engineering, Daffodil International University

Member	Student ID	Role
Md. Shaid Hasan	241-15-360	Team leader: design, implementation, deployment
Md. Fazle Rabbi	241-15-364	Report, public sources on MFS agents, Bangla UI review
Md. Afsahul Arefin Talukder	241-15-377	Demo video, manual QA, clean-clone test

- **Live app:** <https://jogan-bd.vercel.app> (one-click demo analyst or approver on the sign-in page)
- **API:** <https://jogan-api-gt7msysppq-as.a.run.app> (interactive docs at `/docs`)
- **Code:** <https://github.com/imshaid/Jogan> (tag `submission-initial`)
- **Demo video:** https://drive.google.com/file/d/1uXKkTAHMD9jE1exa-kO9ypYZwW8NukoK/view?usp=drive_link

All data in this project is simulated. Jogan is an independent student prototype. It is not an upay product and uses no upay production data.

Abstract

Every mobile financial services (MFS) cash-out needs physical cash at an agent's shop, and every cash-in needs e-float. When either runs out the customer is turned away. In Bangladesh, distributors' runners rebalance agents on a fixed rhythm plus calls, while demand swings with paydays, remittances and Eid. Jogan forecasts each agent's **peak drain** of cash and e-float over the next 6 to 24 hours with calibrated quantile models, turns the forecast into a stock-out chance and a top-up need, plans the runners' morning visits with a mixed-integer program, explains each visit in English and Bangla, and puts every visit in front of a human approver with an append-only audit log. In a seeded simulation of the 2026 Bangladesh calendar, evaluated over seeds never used in development, Jogan turns away fewer customers than the status quo and two stronger rules while driving fewer runner kilometres. It does not win everywhere: it is worse than a simple threshold rule for urban agents, its forecast intervals miss their nominal coverage in several cells, and its anomaly flag mostly misses structuring. Those results are reported next to the wins.

1. Introduction

1.1 The problem

An agent's shop holds two kinds of money. A cash-out drains **physical cash** and adds **e-float** (the agent's digital balance); a cash-in does the reverse. When a side runs dry, the request fails: the customer leaves, the agent loses the commission, and trust in the service suffers. The cost concentrates on predictable days: the start of the month (wages are due within seven working days after the wage period, Labour Act 2006 s.123 [21]), remittance days, and the weeks before Eid-ul-Fitr and Eid-ul-Azha. Bangladesh Bank's monthly MFS statistics show agent cash-out rising sharply in May 2026, the month of Eid-ul-Azha [1, 2]. A single customer may cash out a large amount in one day at an agent point [3], enough to empty a small agent's drawer.

In the format of the Student Guideline (§10):

For upay's liquidity-operations analysts and distributor managers, rebalancing of agents' physical cash and e-float on **fixed rounds and calls**, not on a forecast, **causes** agent stock-outs that turn customers away, costing transactions, agent commission and trust, most of all before Eid. **We will build** Jogan, an AI copilot that **uses** agents' transaction and balance histories **to** forecast each agent's liquidity pressure for the next 6–24 hours and recommend human-approved runner dispatches. **Success is measured by** failed customer requests, known cost and the break-even value of a lost customer versus fixed-round (status quo), threshold and safety-stock policies in a seeded operations simulation.

How big. Bangladesh Bank publishes what agents served each month, not how many customers they turned away, so we size the problem for a range of turned-away rates. Measuring the real rate on upay's own logs is step 0 of the validation plan (§8.2):

Agents served 52.0 crore cash-out and cash-in requests worth ₳89,672 crore in July 2026, across all MFS providers, about 280 a month per agent (1,856,190 agents, February 2025). Bangladesh Bank does not publish how many were turned away for lack of cash or e-float, so each row is a rate, not a measurement:

Share of requests turned away	Customers turned away, July 2026	Value turned away (₳)	Agent commission lost (₳)	Customers turned away, May 2026 (Eid-ul-Azha)	Value turned away (₳)
1%	52.5 lakh	906 crore	3.7 crore	58.1 lakh	1,028 crore
2%	106.1 lakh	1,830 crore	7.5 crore	117.4 lakh	2,077 crore
5%	273.7 lakh	4,720 crore	19.4 crore	302.9 lakh	5,356 crore

1 lakh = 100,000; 1 crore = 10 million. Commission at ₳4.10 per ₳1,000 (`configs/ops/costs.yaml`). Assumptions: turned-away rates are a sensitivity, not a measurement; a turned-away request has the month's average size; published totals are served requests only (turned away = served * $r / (1 - r)$); a customer who comes back later is not netted out. Sources: Bangladesh Bank MFS table 9 and agent count (`configs/calibration/bb_mfs_2026.yaml`). Written by `make sizing` to `artifacts/sizing.json`.

1.2 Users

Role	Need
Liquidity-operations analyst (upay or distributor), primary user	Which agents will run out, of what, and when
Approver (operations or distributor manager)	A short queue of proposed visits with reasons, to approve or reject
Runner (field staff)	An approved route with the amount at each stop (not built)

1.3 Today's practice

A survey of Bangladeshi agents found that almost all rebalance at their shop through distributor runners who visit "usually at a predetermined time", with some distributors also rebalancing on demand [4]. Jogan's status quo is therefore a **fixed round plus calls**, not a forecast. Our hypothesis, to be validated with upay, is that refills are mostly reactive to that rhythm.

The second channel is the shopkeeper's **own trip to a nearby bank**. It works only in bank transaction hours, 10:00 to 15:00 from 5 April 2026 [22], and not on the Friday–Saturday bank weekend or bank holidays; the shop is short-handed or shut while the agent is away, and the cash travels on the street. A drawer that runs dry on a Thursday evening stays dry until Sunday unless a runner comes. The simulator gives every agent this trip under every policy and counts it (§5.1).

1.4 Contributions

1. A seeded simulator of agents, customers and runners on the real 2026 calendar, calibrated to Bangladesh Bank aggregates, with an operations environment that replays every customer against the exact balance.
2. A forecast of the **peak cumulative drain**, not the end-of-window balance, so the stock-out chance is read directly from calibrated quantiles; trained on censoring-corrected labels.
3. A plan that separates prediction from business rules: a newsvendor need from configured costs, then one dispatch program per territory.
4. Explanations from the model's own attributions, templates of record in two languages, and an LLM that may only reword them under numeric checks.
5. An honest evaluation: three baselines, an oracle, paired intervals over unseen seeds, break-even values instead of a guessed customer value, fairness by group, and a list of where Jogan does not win.
6. A live, secured, bilingual prototype with human approval and an audit trail.

2. Approach

2.1 Data strategy: a simulated world

No real agent data was available, and the rules require synthetic data. Jogan's simulator (`jogan/sim`) generates territories, agents and runners from public facts and labelled assumptions (`docs/02-data-assumptions.md`):

- the real 2026 calendar: public holidays, Ramadan, both Eids, the Friday–Saturday bank weekend [5, 6, 7];
- agent cash-in/cash-out levels and ticket sizes calibrated to Bangladesh Bank's monthly tables [2];
- district hubs from an MIT-licensed geocode dataset [8];
- demand patterns: hour of day, weekday, payday, remittance days, hat (market) days, Eid surges, disruptions;
- injected anomalous agents (split cash-outs, unexplained spikes, night activity) to test the anomaly flag.

Every number in the configs is tagged with its source, `DERIVED` or `ASSUMPTION`, and a test enforces the tags. The world's ground truth (every request, including the lost ones) is kept apart from what Jogan may see (served flows, e-float, a cash estimate, with late and missing records).

2.2 Operations environment and baselines

`jogan/ops` replays customers one by one against the agent's balance, with runners, shifts, calls and agents' own bank trips. Every policy meets the **same customers** (common random numbers), so a difference between policies is the policy, not noise. Costs come from sourced inputs only: petrol prices, a commuter motorcycle's

rated mileage, a distribution officer's advertised salary, the agent commission and the policy rate [9–13]. A lost customer's value is unknown, so it is not priced; the evaluation reports the value at which two policies break even, with Fieller's interval [20].

Baselines: **fixed round** (status quo), **threshold** (visit below a cover of typical days), **safety stock** (mean + k-sd of the recent peak drain). The **oracle** sees the future and marks the room left.

2.3 Forecast

- **Target:** the peak cumulative drain of cash and of e-float over 6, 12 and 24 hours from the forecast origin. A stock-out happens when the balance hits zero at any point, so $P(\text{stock-out}) = P(\text{peak drain} > \text{balance now})$.
- **Labels:** during a stock-out demand is hidden, so a censored hour is lifted to served plus expected flow; dropping censored windows instead keeps only quiet ones and biases the model.
- **Model:** LightGBM quantile regression [14], one booster per side, horizon and level, on `log1p`, with leak-free features (a record is used only after it arrived; a test perturbs late records and checks nothing moves).
- **Calibration:** asymmetric conformalized quantile regression [15], one shift per level and agent group, so each upper quantile carries its own guarantee.

2.4 From forecast to a plan

- **Newsvendor need** per side at the critical ratio of underage (lost commission plus the operator's lost-customer value) to overage (the cost of idle money).
- **Value of a visit:** the expected shortage cost avoided plus the call trip it saves.
- **Dispatch:** one mixed-integer program per territory, a generalized-assignment heuristic for routing [16] solved by HiGHS [17], maximising value minus fuel and runner time under visit and shift limits, with a route check, a fill step and a greedy fallback.

The model predicts; the rules and the optimizer decide what to propose; a human decides what happens.

2.5 Explanations, guardrails and the LLM

- **Drivers:** exact TreeSHAP [18] from LightGBM's `pred_contrib`, top three per visit.
- **Template of record:** English and Bangla text built only from the stored evidence (Bangla digits, lakh grouping, ষ).
- **Gemini** (`gemini-3.5-flash-lite`, fallback `gemini-3.1-flash-lite`, free tier, simulated data only) may reword a template on request. The rewording is refused if it adds a number, drops the stock-out chance, uses the wrong script or is too long; no user text ever enters the prompt.
- **Guardrails** send a visit to manual review when the agent is outside the training range, the interval is wide, data is missing, history is short or the anomaly flag fired. Approving a flagged visit needs a note.
- **Anomaly flag:** an Isolation Forest per setting [19] plus two rules, advisory only.

2.6 Human approval and audit

Two roles: an analyst reads, an approver decides. The decision and its audit row are written in one database transaction; triggers keep the audit log append-only even for the table owner. Every visit has an 8-step trace from forecast to human decision, each step naming who produced it and the config hash it ran under.

3. Implementation process

3.1 Milestones

The project was built during the event, in small commits, following a milestone plan with a written logic chain before heavy coding:

Milestone	Delivered
M0	Repository, uv project, lint, tests, CI, secret scanning, Dependabot, decision log
M1	Logic chain, requirements checklist, data assumptions with verified sources
M2	World simulator, calibration, profiles, tests
M3	Operations environment, baselines, status-quo history log
M4	Features with a leakage test, quantile forecast, CQR, backtest, censoring
M5	Newsvendor and dispatch program, multi-seed comparison, ablations, fairness, make eval
M6	Walking skeleton live: Cloud Run, Vercel, Supabase schema with RLS and audit
M7	Explanations, guardrails, Gemini narrator, anomaly flag
M8	Full API: token checks, roles, rate limits, validation, decision trace
M9	Web interface: map, queue, agent detail, impact, audit, about; Bangla and English
M10	Final evaluation check, stress test, monitoring, keep-alive
M11	README, documentation pack, this report, video script
M12	Clean-clone test, live check, submission tag

3.2 Decisions that changed the design

The decision log ([docs/DECISIONS.md](#)) records each choice with its reason. The ones that changed the outcome:

- **Environment and baselines before ML** (D-002). The training data is itself the log of today's policy, stock-outs included, and the baselines set the bar early.
- **Deploy a walking skeleton early** (D-002, D-022). Cloud Run, Vercel and Supabase were live before the interface was built, so deployment problems surfaced while there was time.
- **The status quo was wrong at first** (D-016). The first run used agents' own bank trips as the status quo and lost far more requests than Bangladeshi agents report. The agent survey [4] showed that runners on a fixed rhythm are the norm, so the status quo became a fixed round plus calls.
- **No made-up costs** (D-019). Per-km and per-visit guesses were replaced by costs built from sourced inputs, and the unknown value of a lost customer became a break-even value instead of a number we chose.
- **Censoring matters** (D-020). Served flows understate demand exactly in the windows that matter; estimated-demand labels fixed most of the bias.
- **The textbook split lost** (D-021). The two-sided newsvendor split of liquidity lost more requests than the simple split in proportion to typical outflow, so Jogan uses the latter and keeps the former as an ablation. Adding the saved call trip to a visit's value stopped the program from skipping visits that came back as calls.
- **A guardrail that always fires is not a guardrail** (D-023). "Cash plus e-float cannot cover both needs" fired on nearly every visited agent and was dropped.
- **Rate limits must not trust the client** (D-024). The leftmost X-Forwarded-For entry is written by the client; the guard reads the header from the right, verified on the live service from two networks.

- **Keep-alive needs the database** (D-026). A free Supabase project pauses after a week without database activity, and the uptime monitor cannot send an API key header, so the API got a `/health/db` route that runs one query.

3.3 Engineering practice

- **Configuration over code:** every tunable number in tagged YAML; config hashes stored with every result and recommendation.
- **Tests:** the simulator's patterns, the environment's invariants (liquidity is conserved by a visit), forecast leakage and determinism, the planner, explanations, guardrails, the anomaly flag, the API with real signed tokens and forged ones, rate limits and validation; database rules for every role on Postgres 17; a live end-to-end check in a real browser.
- **Numbers never typed by hand:** `make eval` writes `artifacts/metrics.json`; `make impact` and `make docs` copy it into the web app and the documents; tests fail if a copy is stale.
- **Reproducibility:** seeded everything, deterministic LightGBM, the demo bundle rebuilt from a seed inside the Docker image, no model binaries in git.
- **Security from the start:** secrets in Secret Manager, keyless deploys, gitleaks, least-privilege accounts.

4. System and key features

simulator → operations environment → observed log → forecast + CQR → newsvendor + dispatch
→ drivers, guardrails, templates → served bundle → API (Cloud Run) ↔ Supabase (RLS, audit)
→ web app (Vercel): map, queue, agent detail, impact, audit, about

4.1 Key features

Feature	What the user gets
Network map	Every agent on a map, shaped by its stock-out risk band (▲ high, ◆ medium, ● low; never colour alone), a ring for each planned visit, a timeline of planned visits, and the same facts as territory and "highest risk" tables
Visit queue	Each morning's recommended runner visits, highest value first, with filters; a "Why?" row with the explanation, the drivers and any manual-review reason
Human approval	An approver approves or rejects each visit; a flagged visit needs a note; an analyst can read everything and decide nothing
Agent detail and decision trace	One agent's stock-out chance over the test window, the forecast drain against the balance, and the 8-step trace from forecast to human decision
English and Bangla	The whole interface and every explanation in both languages, with Bangla digits, lakh grouping and ত
AI rewording	On request, Gemini rewords the template explanation; labelled AI-written with the model id, and discarded if it adds a number
Anomaly flags	Advisory flags on unusual agent-days that send the visit to manual review
Audit log	Every decision with who, when, the manual-review mark and the note; append-only
Impact page	The evaluation in the browser, with fairness by group and where Jogan does not win (public, no sign-in)
Local run	<code>make run</code> starts the API and the web app with local sign-in, no accounts needed

Every output is labelled Prediction, Template, AI-written, Assumption or Evaluation, and a "Simulated data" badge is always visible.

4.2 Technology

Layer	Technology
ML and optimization	Python 3.12, NumPy, pandas, LightGBM, scikit-learn, SciPy (milp, HiGHS)
API	FastAPI on Google Cloud Run (Singapore), PyJWT
Database and auth	Supabase Postgres and Auth, row-level security, SQL functions, triggers
Web	Next.js, React, Tailwind CSS, MapLibre GL JS with OpenFreeMap tiles
LLM	Gemini via Google AI Studio (rewording only)
Delivery	GitHub Actions (CI, deploy, keep-alive), Vercel, Docker, Dependabot, gitleaks

Details: [docs/03-architecture.md](#) and [docs/09-deployment.md](#).

5. Results

5.1 Service and cost

Policy	Lost requests per 1,000	Lost requests	Runner km	Known cost (₹)
Fixed round (status quo)	118.4 (115.9 to 120.9)	19,376 (18,935 to 19,818)	41,861 (41,689 to 42,034)	713,522 (708,066 to 718,979)
Threshold	111.1 (108.4 to 113.8)	18,188 (17,707 to 18,668)	41,848 (41,426 to 42,270)	699,550 (693,828 to 705,273)
Safety stock	111.5 (108.7 to 114.3)	18,246 (17,771 to 18,720)	41,481 (41,123 to 41,839)	706,256 (699,882 to 712,631)
Jogan	108.9 (106.1 to 111.6)	17,820 (17,338 to 18,300)	40,194 (39,776 to 40,612)	693,700 (687,341 to 700,058)
Oracle (perfect foresight, not deployable)	89.0 (86.4 to 91.5)	14,565 (14,094 to 15,035)	43,496 (43,126 to 43,867)	658,728 (652,808 to 664,648)

Jogan minus each baseline, paired by seed (negative means Jogan is lower):

Baseline	Lost per 1,000	Runner km	Known cost (₹)	Break-even
Fixed round (status quo)	-9.51 (-10.07 to -8.96)	-1,667 (-2,001 to -1,334)	-19,823 (-22,859 to -16,787)	Jogan cheaper at every value of a lost customer
Threshold	-2.25 (-2.69 to -1.81)	-1,654 (-1,883 to -1,425)	-5,851 (-7,463 to -4,238)	Jogan cheaper at every value of a lost customer
Safety stock	-2.61 (-3.15 to -2.06)	-1,287 (-1,426 to -1,148)	-12,557 (-14,071 to -11,043)	Jogan cheaper at every value of a lost customer

In the Eid-ul-Azha window (2026-05-18 to 2026-05-31), Jogan minus the best baseline (Threshold): -1.94 (-2.43 to -1.44) lost requests per 1,000.

Known cost is runner time and fuel, lost commission and idle liquidity at the middle runner salary (`configs/ops/costs.yaml`); Jogan runs at a lost-customer value of ₹20. The oracle sees the future and only marks how much room is left.

Evaluation on simulated data: profile `full`, 10 seeds (1000 to 1009), test window 2026-05-07 to 2026-06-03 (28 days), mean and 95% interval over seeds. Source: `artifacts/metrics.json`, written by `make eval`.

How often agents run dry, what it is worth, and their own bank trips. The simulator knows every customer turned away, what they asked for, and every time an agent went to a bank to refill:

Test window	Fixed round (status quo)	Threshold	Safety stock	Jogan
Requests turned away per 1,000	118.4 (115.9 to 120.9)	111.1 (108.4 to 113.8)	111.5 (108.7 to 114.3)	108.9 (106.1 to 111.6)
Requests turned away	19,376 (18,935 to 19,818)	18,188 (17,707 to 18,668)	18,246 (17,771 to 18,720)	17,820 (17,338 to 18,300)
Agent-days with a customer turned away	45.6%	44.3%	44.1%	43.5%
Value turned away (₹)	74,511,295	72,232,235	72,483,650	71,303,785
of it cash-out (₹)	37,666,940	35,760,725	36,084,595	34,633,215
Agent commission lost (₹)	305,496	296,152	297,183	292,346
Agents' own bank trips	1,134 (1,109 to 1,160)	932 (905 to 959)	930 (901 to 958)	922 (894 to 950)

Agents' own bank trips, Jogan minus each baseline, paired by seed: Fixed round (status quo) -212.6 (-232.1 to -193.1); Threshold -9.9 (-22.2 to 2.4); Safety stock -7.8 (-27.9 to 12.3). An agent goes to a bank only on a bank open day and in bank hours (`configs/ops/env.yaml`), so a drawer that runs dry on a Friday, a Saturday, a holiday or after the bank closes stays dry until a runner comes.

Evaluation on simulated data: profile `full`, 10 seeds (1000 to 1009), test window 2026-05-07 to 2026-06-03 (28 days), mean and 95% interval over seeds. Source: `artifacts/metrics.json`, written by `make eval`.

5.2 Hypotheses

The hypotheses were written in the logic chain before the evaluation ran:

	Hypothesis	Result
H1	cheaper at every value, or break-even reported with an interval	at ₳20: Fixed round (status quo): Jogan cheaper at every value of a lost customer; Safety stock: Jogan cheaper at every value of a lost customer; Threshold: Jogan cheaper at every value of a lost customer
H2	fewer failed requests than the best baseline without more runner km	holds: against Threshold, -2.25 (-2.69 to -1.81) lost per 1,000 and -1,654 (-1,883 to -1,425) km
H3	interval coverage within ± 5 points of nominal, overall and by group	fails: 45 cells outside ± 5 points
H4	no agent group significantly worse served than under the best baseline	fails: worse in <code>setting=urban</code> , <code>territory=DHK</code>

5.3 Forecast quality

Side	Horizon	Pinball loss, ₳: Empirical / Naive + CQR / LightGBM / LightGBM + CQR	90% interval: coverage (mean width)	Brier: Empirical / LightGBM + CQR	Stock-out base rate
Cash	6 h	654 / 765 / 585 / 582	90.3% (₳5,920)	0.089 / 0.084	14.1%
Cash	12 h	856 / 959 / 744 / 740	89.7% (₳7,291)	0.105 / 0.097	18.0%
Cash	24 h	1,404 / 1,406 / 1,216 / 1,207	87.7% (₳11,400)	0.157 / 0.147	29.2%
E-float	6 h	654 / 776 / 598 / 595	91.0% (₳6,615)	0.071 / 0.069	10.0%
E-float	12 h	846 / 953 / 749 / 744	90.2% (₳8,239)	0.088 / 0.085	13.4%
E-float	24 h	1,307 / 1,317 / 1,119 / 1,112	87.5% (₳12,278)	0.135 / 0.127	21.5%

Pinball loss is averaged over the eight quantile levels and scored against true demand (lower is better). Coverage is of the calibrated LightGBM interval against true demand (nominal 90%). Brier scores the stock-out chance for the no-top-up event (lower is better).

Evaluation on simulated data: profile `full`, 10 seeds (1000 to 1009), test window 2026-05-07 to 2026-06-03 (28 days), mean and 95% interval over seeds. Source: `artifacts/metrics.json`, written by `make eval`.

5.4 Fairness

Group	Value	Jogan lost per 1,000	Threshold lost per 1,000	Jogan minus best	Verdict
setting	<code>peri_urban</code>	75.5 (72.9 to 78.1)	77.4 (74.9 to 80.0)	-1.95 (-2.27 to -1.63)	lower
setting	<code>rural</code>	156.9 (151.8 to 162.0)	161.9 (157.0 to 166.8)	-5.03 (-6.50 to -3.56)	lower

Group	Value	Jogan lost per 1,000	Threshold lost per 1,000	Jogan minus best	Verdict
setting	urban	121.8 (116.5 to 127.1)	120.8 (115.5 to 126.1)	1.03 (0.01 to 2.06)	higher
size_class	large	64.9 (59.4 to 70.4)	68.6 (62.9 to 74.3)	-3.68 (-4.50 to -2.86)	lower
size_class	medium	96.6 (93.6 to 99.7)	98.3 (95.2 to 101.3)	-1.63 (-2.42 to -0.85)	lower
size_class	small	180.7 (176.5 to 184.9)	182.0 (177.9 to 186.2)	-1.33 (-2.84 to 0.18)	no significant difference
territory	CUM	80.7 (74.5 to 86.9)	83.7 (77.8 to 89.5)	-2.95 (-4.21 to -1.70)	lower
territory	DHK	121.8 (116.5 to 127.1)	120.8 (115.5 to 126.1)	1.03 (0.01 to 2.06)	higher
territory	GZP	68.7 (63.2 to 74.1)	68.9 (63.8 to 74.1)	-0.28 (-1.23 to 0.67)	no significant difference
territory	KUR	156.4 (150.2 to 162.5)	160.6 (155.2 to 165.9)	-4.20 (-6.04 to -2.36)	lower
territory	RNG	157.9 (150.8 to 165.1)	163.8 (155.8 to 171.8)	-5.88 (-7.93 to -3.83)	lower
territory	SYL	77.9 (73.9 to 81.8)	80.4 (76.3 to 84.5)	-2.52 (-3.45 to -1.59)	lower

5.5 Ablations

Jogan minus ...	Lost per 1,000	Runner km	Known cost (₹)
Greedy round instead of the MILP (same forecast and levels)	-0.72 (-1.22 to -0.22)	-1,340 (-1,499 to -1,181)	-7,993 (-9,161 to -6,824)
Two-sided newsvendor split instead of the typical split	-2.72 (-3.22 to -2.22)	-105 (-302 to 92)	-6,662 (-8,027 to -5,297)

5.6 Anomaly flag

Injected pattern	Windows	With a flag
Night activity	6	6
Unexplained spike	6	3
Split cash-outs	9	3

168,000 test agent-days; 1,196 flagged (0.7%), 51 of them on injected anomalies: precision 4.3% against a base rate of 0.08%. Precision at 5 / 10 / 20 per seed, averaged: 0.24 / 0.36 / 0.22. Injected windows with at least one flag: 12 of 21. Of the 346 injected split cash-outs in the test windows, 18 (5%) were served; the rest found the drawer short and were turned away, which leaves no record for the flag to see.

5.7 Scale

Measure	Mean	Max
One morning, end to end (s)	3.1	4.3
Plan (s)	2.4	3.5
of which MILP solver (s)	1.7	2.8
Drivers, guardrails, flags (s)	0.7	0.8

10,000 agents, 260 runners, 60 territories; 7 Jogan mornings after 7 days of status quo. 360 programs, 0 fallbacks, 23,836 visits; peak memory 1,779 MB; whole run 101 s.

Timing only, on 13th Gen Intel(R) Core(TM) i7-13650HX (20 threads). Source: `artifacts/stress.json`, written by `make stress`.

6. Where Jogan does not win

- **Some groups are served worse than by the best baseline (Threshold).** Lost requests per 1,000, Jogan minus Threshold: `urban` and `DHK` (the same agents) 1.03 (0.01 to 2.06).
- **Forecast intervals are off their nominal coverage by more than 5 points in 45 cells** (by side, horizon, interval and agent group), 4 of them over all agents.
- **The oracle is still ahead:** Jogan minus the oracle, 19.9 (19.1 to 20.7) lost requests per 1,000.
- **The anomaly flag is weak on structuring:** split cash-outs found in 3 of 9 injected windows; precision 4.3% against a base rate of 0.08%.
- **62 group-level comparisons** (across lost-customer values and baselines) show no significant win or a baseline as good or better (`does_not_win` in `artifacts/metrics.json`).

Group-level cases at Jogan's default setting:

Baseline	Agent group	Jogan minus baseline, lost per 1,000	Why no win
Fixed round (status quo)	<code>setting = urban</code>	-1.73 (-3.50 to 0.05)	no significant difference
Fixed round (status quo)	<code>territory = DHK</code>	-1.73 (-3.50 to 0.05)	no significant difference
Threshold	<code>setting = urban</code>	1.03 (0.01 to 2.06)	baseline as good or better
Threshold	<code>size_class = small</code>	-1.33 (-2.84 to 0.18)	no significant difference
Threshold	<code>territory = DHK</code>	1.03 (0.01 to 2.06)	baseline as good or better
Threshold	<code>territory = GZP</code>	-0.28 (-1.23 to 0.67)	no significant difference
Safety stock	<code>setting = urban</code>	0.69 (-0.19 to 1.56)	baseline as good or better
Safety stock	<code>size_class = small</code>	1.17 (-0.23 to 2.57)	baseline as good or better
Safety stock	<code>territory = DHK</code>	0.69 (-0.19 to 1.56)	baseline as good or better
Safety stock	<code>territory = GZP</code>	-0.80 (-1.78 to 0.18)	no significant difference

At Jogan's setting of `ᵇ20`. Over every setting: 31 no significant difference, 31 baseline as good or better (62 in all).

Also not in Jogan's favour, or not measured:

- **Costs left unpriced** for lack of a source: motorcycle wear, the runner's phone, the agent's own time on bank trips.
- **Not modelled in the plan:** the runner's bag inside the program, the hours between the morning forecast and the runner's arrival, and a call that rescues an agent who was not visited.
- **Hourly resolution:** a dip inside an hour is not seen by the target.
- **Eid day itself:** no runner is on duty, so no policy can help.

7. Responsible AI and security

- Synthetic data only, and only agent-level aggregates by design; nothing personal reaches the LLM.
- A human approves or rejects every visit; flagged visits need a written note; the audit log cannot be edited.
- Roles are enforced twice, by the API (which verifies every token against Supabase's keys) and by row-level security; rate limits, strict validation and one error body.

- Every output is labelled Prediction, Template, AI-written, Assumption or Evaluation; risk is never shown by colour alone.
- Fairness by group is measured and its failures reported.

For a pilot on real data (on-site addition, D-037), the same document defines an access-control matrix (who may read, decide, publish or change what, and what enforces it), how each kind of agent data is classified and protected (no customer data is ever ingested; balances, locations and anomaly flags are confidential), the model monitoring run each morning (interval coverage per group, Brier score against a simple forecast, data gaps, input drift, the approvers' override rate, the anomaly flag rate, optimizer fallbacks), each with an alert threshold and an action, and the override and escalation steps, ending in a kill switch back to fixed rounds.

Details and the threat table: [docs/06-responsible-ai.md](#).

8. Intended impact and path to product

8.1 Intended real-life impact

- **Customers:** cash-outs and cash-ins that work when they are needed, above all on paydays, remittance days and before Eid, when a turned-away customer hurts most.
- **Agents:** fewer lost commissions, and a runner who comes before the drawer or the e-float runs dry rather than after a call.
- **Distributors:** runner time and fuel spent where a visit avoids the most failed requests, and a short queue to approve instead of a fixed round.
- **upay:** more successful transactions and more trust in the service, with every recommendation traceable and every decision audited, from agent-level aggregates only (no customer data).
- **Measured the same way in a pilot:** failed requests per 1,000, runner km and visits, agent commission, and each of them per agent group.

In business terms (on-site addition, D-033), from the same evaluation runs:

KPI (Jogan minus baseline; negative is a saving)	vs status quo, 28-day test window	vs status quo, per 1,000 agents a month	vs Threshold, per 1,000 agents a month
Failed transactions	-1,557 (-1,644 to -1,469)	-2,780 (-2,936 to -2,623)	-657 (-789 to -526)
Transaction value turned away (₹)	-3,207,510 (-3,523,530 to -2,891,490)	-5,727,696 (-6,292,018 to -5,163,375)	-1,657,946 (-2,049,604 to -1,266,289)
of it cash-out (₹)	-3,033,725 (-3,176,928 to -2,890,522)	-5,417,366 (-5,673,087 to -5,161,646)	-2,013,411 (-2,225,843 to -1,800,979)
Agent commission lost (₹)	-13,151 (-14,446 to -11,855)	-23,484 (-25,797 to -21,170)	-6,798 (-8,403 to -5,192)
Runner km	-1,667 (-2,001 to -1,334)	-2,977 (-3,572 to -2,382)	-2,953 (-3,362 to -2,544)
Runner cost, time and fuel (₹)	-5,752 (-7,967 to -3,537)	-10,271 (-14,227 to -6,316)	-2,822 (-5,080 to -564)
Agents' own bank trips	-213 (-232 to -193)	-380 (-414 to -345)	-18 (-40 to 4)
Known cost (₹)	-19,823 (-22,859 to -16,787)	-35,398 (-40,820 to -29,976)	-10,448 (-13,327 to -7,568)

Return on investment. Against the status quo, Jogan's known cost is lower by ₹35,398 per 1,000 agents a month (95% interval ₹29,976 to ₹40,820), before any value is put on a customer kept. So it pays for itself while running it (cloud, an analyst and an approver's time) costs less than that; every customer kept is extra. The running cost was not measured.

Paired by seed; scaled from the simulated network of 600 agents to 1,000 agents and 30 days. Known cost is runner time and fuel, lost commission and idle liquidity at the middle runner salary. Transaction value turned away is what customers asked for and did not get; upay's own fee on it is not public and is not priced.

Evaluation on simulated data: profile `full`, 10 seeds (1000 to 1009), test window 2026-05-07 to 2026-06-03 (28 days), mean and 95% interval over seeds. Source: `artifacts/metrics.json`, written by `make eval`.

8.2 Path to product

The forecast reads one documented table that upay's ledger can produce from agent-hour aggregates; storage sits behind one interface; sign-in maps to two roles. Validation would go: first measure the problem on upay's own logs (how often agents sit with too little cash or e-float, the requests and commission turned away in those hours with the same censoring correction the forecast uses, and agents' own bank trips, checked against a short manual tally at sample agents), then a backtest on upay history, then shadow mode (Jogan plans, nobody acts), then a randomised pilot by distributor territory with go/no-go thresholds agreed in advance, measuring failed requests (e-float exactly from the ledger, cash through a proxy such as an agent's "could not serve" button), runner km and agent commission per group. Before a pilot: an equity floor in the optimizer, the runner's bag and arrival time in the program, a runner route view, and an LLM provider approved by upay or templates only. Details: [docs/07-product-readiness.md](#).

9. Conclusion

A calibrated peak-drain forecast plus a cost-aware dispatch program turns away fewer customers than the status quo and the best simple rule in a simulated Bangladesh, with fewer runner kilometres, while keeping a person in charge of every visit and a record of every decision. The margin over the best rule is modest, it reverses for urban agents, and every number comes from a simulator built on public facts. The next step is not a bigger model but real data: a backtest on upay's history and a shadow run.

10. AI usage

Jogan was built with Claude Code (Claude Opus 5.5) under the team's review; commits made with it carry a co-author trailer, and the prompt history is available on request. Inside the product, Gemini only rewords explanations. Full disclosure: [docs/08-ai-usage.md](#).

References

1. Bangladesh Bank, *MFS account information* (Table 8).
https://www.bb.org.bd/econdata/fin_digitalfstat/tab8.pdf
2. Bangladesh Bank, *MFS transaction statistics* (Table 9).
https://www.bb.org.bd/econdata/fin_digitalfstat/tab9.pdf
3. BSS, Bangladesh Bank circular of 27 March 2025 on MFS customer limits.
<https://www.bssnews.net/business/258749>
4. Helix Institute of Digital Finance / MicroSave, *Agent Network Accelerator Survey: Bangladesh Country Report*, 2014. <https://www.microsave.net/wp-content/uploads/2014/11/Agent-Network-Accelerator-Bangladesh-Country-Report-2014.pdf>
5. The Daily Star, official list of public holidays 2026.
<https://www.thedailystar.net/news/bangladesh/news/govt-announces-official-list-public-holidays-2026-4031596>

6. BSS (Eid-ul-Fitr 2026) <https://www.bssnews.net/news/370394>; Ittefaq (Eid-ul-Azha 2026) <https://en.ittefaq.com.bd/16594/bangladesh-to-celebrate-eid-ul-azha-on-may-28>
7. The Financial Express, banks' weekly holidays. <https://thefinancialexpress.com.bd/views/opinions/banking-on-holidays>
8. nuhil/bangladesh-geocode (MIT). <https://github.com/nuhil/bangladesh-geocode>
9. Fuel prices 2026: The Financial Express <https://thefinancialexpress.com.bd/trade/fuel-prices-cut-by-tk-2-a-litre-at-start-of-2026>; Dhaka Tribune <https://www.dhakatribune.com/bangladesh/power-energy/420367/fuel-prices-rise-up-to-nearly-36%25-between-january>; TBS <https://www.tbsnews.net/node/1426246>; UNB <https://unb.com.bd/category/Bangladesh/fuel-prices-raised-again-octane-petrol-up-by-tk-5-per-litre/187098>
10. BikeBD, Bajaj Platina 100 specifications. <https://bikebd.com/price/bajaj-platina-100-2015>
11. Distribution sales officer job ads, 2026: EZ Jobs https://ezjobsbangla.com/jobs/bkash-distribution-sales-officer--j_wE-olwkZQKe2FRlqJ-UF6A; Niyog <https://niyog.co/jobs/bkash-distribution-sales-officer-883a8e53>
12. Prothom Alo, MFS agent commission, 10 August 2022. <https://www.prothomalo.com/business/7cxvrytmp6>
13. The Financial Express, Bangladesh Bank policy rate. <https://thefinancialexpress.com.bd/economy/bb-cuts-repo-rate-by-50-bps-to-950pc-to-spur-investment-economic-recovery>
14. G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree", *NeurIPS*, 2017.
15. Y. Romano, E. Patterson and E. Candès, "Conformalized quantile regression", *NeurIPS*, 2019. <https://arxiv.org/abs/1905.03222>
16. M. L. Fisher and R. Jaikumar, "A generalized assignment heuristic for vehicle routing", *Networks* 11(2), 109–124, 1981.
17. Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method", *Mathematical Programming Computation* 10, 119–142, 2018 (HiGHS).
18. S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees", *Nature Machine Intelligence* 2, 56–67, 2020.
19. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest", *IEEE ICDM*, 2008.
20. E. C. Fieller, "Some problems in interval estimation", *Journal of the Royal Statistical Society B* 16(2), 175–185, 1954.
21. Bangladesh Labour Act 2006, Chapter IX (working hours, wages). <https://www.lawyersnjurists.com/article/the-bangladesh-labour-act-2006-chapter-ix/>; wage timing (s.123): The Daily Star <https://www.thedailystar.net/law-our-rights/news/the-entitlements-the-workers-relating-wages-1890601>
22. Bank transaction hours 10:00–15:00 from 5 April 2026: Dhaka Tribune <https://www.dhakatribune.com/business/banks/406921/bb-reschedules-bank-transaction-hours>

Appendix: reproducing the results

```
git clone https://github.com/imshaid/Jogan.git && cd Jogan
make setup && make check
make eval      # artifacts/metrics.json
make docs      # the numbers in this report
```

`make eval` took 26 min for 10 seeds (`meta.runtime_s`). `make stress` took 101 s at a peak of 1,779 MB on 13th Gen Intel(R) Core(TM) i7-13650HX (20 threads, 15 GB RAM).